

Reproducibility, dataset splitting and global constants

Important: To avoid data leakage, all dataset partitioning is performed at the original *track* level before any slicing into shorter segments. In other words, the 1000 GTZAN audio files are first split (track-wise, stratified by genre) into train/val/test sets; only after this split we generate 3-second non-overlapping slices from the tracks belonging to each partition. This guarantees that segments coming from the same original audio file cannot appear in both training and evaluation sets.

The next code cell defines the global feature-extraction and training constants used throughout the notebooks (sample rate, STFT parameters, number of Mel bands, SpecAugment policy, training hyperparameters and random seed). Please edit values if you changed experiment settings.

00 — Setup & Data Preparation (GTZAN)

Replica del notebook originale di preprocessing con path robusti e salvataggi in `data/processed`.

```
In [1]: # Import & Paths
import os, sys, numpy as np, pickle
from pathlib import Path
PROJECT_ROOT = Path(os.getcwd()).resolve().parents[1]
DATA_PATH = PROJECT_ROOT / 'data' / 'gtzan' / 'genres'
PROCESSED_DATA_PATH = PROJECT_ROOT / 'data' / 'processed'
PROCESSED_DATA_PATH.mkdir(parents=True, exist_ok=True)
print('PROJECT_ROOT:', PROJECT_ROOT)
print('DATA_PATH:', DATA_PATH)
print('PROCESSED_DATA_PATH:', PROCESSED_DATA_PATH)
```

```
PROJECT_ROOT: /home/alepot55/Desktop/projects/naml_project
DATA_PATH: /home/alepot55/Desktop/projects/naml_project/data/gtzan/genres
PROCESSED_DATA_PATH: /home/alepot55/Desktop/projects/naml_project/data/proce
ssed
```

```
In [2]: # Reuse logic by loading arrays already produced by the original 00 notebook
import numpy as np
files = ['X_train.npy', 'y_train.npy', 'X_val.npy', 'y_val.npy', 'X_test.npy', 'y_test.npy']
missing = [f for f in files if not (PROCESSED_DATA_PATH / f).exists()]
if missing:
    raise FileNotFoundError(f'Mancano file processati: {missing}. Esegui pri
else:
    X_train = np.load(PROCESSED_DATA_PATH/'X_train.npy'); y_train = np.load(PROCESSED_DATA_PATH/'y_train.npy')
    X_val = np.load(PROCESSED_DATA_PATH/'X_val.npy'); y_val = np.load(PROCESSED_DATA_PATH/'y_val.npy')
    X_test = np.load(PROCESSED_DATA_PATH/'X_test.npy'); y_test = np.load(PROCESSED_DATA_PATH/'y_test.npy')
```

```
with open(PROCESSED_DATA_PATH/'label_encoder.pkl','rb') as f: label_encd
print('Dati caricati correttamente:', X_train.shape, X_val.shape, X_test
```

Dati caricati correttamente: (6000, 128, 128, 1) (2000, 128, 128, 1) (2000, 128, 128, 1)